

Computer Vision

Summer of Science 2026

End-Term Report

Tanmay Dhule

Roll No. 25B4208

Mentor: Sandipan Dhali

GitHub Repository

github.com/tanmayy9/Virtual-Mouse-SOS-26

Indian Institute of Technology, Bombay

May 2026 – July 2026

Contents

1	Introduction	4
1.1	Project Overview and Timeline	4
1.2	Report Structure	4
2	Python Data Structures and NumPy Foundations	5
2.1	Core Data Structures	5
2.2	NumPy	6
3	OpenCV Fundamentals	7
3.1	Reading Images, Video, and Webcam Streams	7
3.2	Basic Image Processing Pipeline	7
3.3	Resizing, Cropping, and Drawing	8
3.4	Displaying Multiple Images (stackImages)	8
3.5	Perspective Transformation	9
3.6	HSV Color Space and Live Color Masking	10
3.7	Contour Detection and Shape Analysis	10
4	Midterm Deliverable: Real-Time Colored Object Tracker	11
4.1	Webcam Setup and Trackbar Windows	11
4.2	Finding and Drawing the Object	12
4.3	Main Loop	12
4.4	Tuning Session and Final Result	13
5	Neural Network Foundations	15
5.1	The Neuron and Activation	15
5.2	Cost Function and Gradient Descent	16
5.3	Backpropagation	17
5.4	Neural Network Variants	18
6	Large Language Models and the Transformer Architecture	18
6.1	What an LLM Is	18
6.2	Data Flow Through a Transformer	18
6.3	Embeddings	19
6.4	The Attention Mechanism	19
6.5	MLP Blocks and Softmax	19
6.6	GPT-3 Parameter Breakdown	20
6.7	Superposition	20
7	Generative Models for Images	20
7.1	CLIP: Bridging Text and Vision	20
7.2	Diffusion Models	21
7.3	Deterministic Sampling: DDIM	21
8	Convolutional Neural Networks	21
8.1	From Linear Classifiers to Neural Networks	22
8.2	Loss Functions	22

8.3	CNN Architecture and Motivation	23
8.4	The Convolutional Layer	24
8.5	Spatial Arrangement	24
8.6	Real-World Case: AlexNet (2012)	25
9	MediaPipe Hands: Real-Time Landmark Detection	25
9.1	Pipeline Overview	25
9.2	The 21 Landmarks	25
9.3	Setting Up the Detector	26
10	End-Term Deliverable: Gesture-Controlled Virtual Mouse	27
10.1	Setup and Configuration	27
10.2	Reading Landmarks and Detecting Extended Fingers	28
10.3	Pinch-to-Click with Debouncing	28
10.4	Cursor Mapping and Smoothing	29
10.5	Scroll Mode	30
10.6	Main Loop	31
10.7	Live Demo	31
11	Conclusion	32
	References	33

1 Introduction

1.1 Project Overview and Timeline

The Computer Vision track of Summer of Science 2026 runs from May to July under the mentorship of Sandipan Dhali. The goal is to build a practical, end-to-end foundation in computer vision — from Python and NumPy fundamentals through core OpenCV operations — culminating in a real-time colored object tracker, alongside an independent study into neural networks, transformers, and generative models. The weekly structure is as follows:

Week 1 (May 24–30): Python basics and environment setup — variables, loops, functions, lists, NumPy arrays; understanding a digital image as a grid of pixel values; installing OpenCV and reading the first image in code.

Week 2 (May 31–June 6): Core OpenCV operations — color spaces, resizing, cropping, drawing shapes, Gaussian blur, thresholding, and Canny edge detection; processing a live webcam feed for the first time.

Week 3 (June 7–13): Contour detection, bounding boxes, and morphological operations (erosion, dilation); mini-project — a real-time colored object tracker using HSV masking.

Midterm Submission (3rd week of June): A working real-time colored object tracker with documented OpenCV code, demonstrating comfort with pixel-level operations, color spaces, and live video processing.

Week 4 (June 14–20): Introduction to machine learning and neural networks — supervised learning, loss functions, gradient descent, and backpropagation; extended independently into transformers and generative models.

Week 5 (June 21–27): Convolutional Neural Networks — convolution filters, pooling, feature maps, and why CNNs outperform MLPs on images, following Stanford CS231n and Andrew Ng’s Deep Learning Specialization.

Week 6 (June 28–July 4): MediaPipe Hands — understanding and integrating the pre-trained 21-point hand landmark detection pipeline with OpenCV.

Week 7 (July 5–11): Virtual mouse core build — mapping fingertip coordinates to screen space, cursor control via `pyautogui`, and pinch-based click gestures.

Week 8 (July 12–18): Polish and documentation — cursor smoothing, gesture reliability, scroll mode, and final report.

End-Term Submission (3rd week of July): A fully functional gesture-controlled virtual mouse using OpenCV and MediaPipe, with documented codebase and a live demo.

1.2 Report Structure

Section 2 covers the Python data structures and NumPy operations used throughout the project. Section 3 documents the core OpenCV toolkit built across Weeks 2–3. Section 4

presents the midterm deliverable — the real-time colored object tracker. Section 5 is an independent-study supplement on neural network foundations. Section 8 covers Convolutional Neural Networks (Week 5). Section 9 introduces the MediaPipe Hands pipeline (Week 6). Section 10 presents the end-term deliverable — the gesture-controlled virtual mouse — with a full code walkthrough and live demo. Sections on transformers and generative image models, an independent extension beyond the formal syllabus, close out the report before the conclusion.

2 Python Data Structures and NumPy Foundations

Week 1 covered Python’s core built-in data structures and the NumPy array library that OpenCV is built on top of. This section documents both, with the small scripts used to learn each concept.

2.1 Core Data Structures

Lists are ordered, mutable, heterogeneous collections. Slicing uses `[start:stop:step]`, and any of the three components can be omitted. This is the same mechanism used to crop a NumPy image array, extended to two or three dimensions:

Listing 1: List operations and slicing

```
1 nums = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]
2 nums.append(10); nums.sort()
3 print(nums[2:5])      # [2, 3, 4]      -- index 2 up to (not incl.) 5
4 print(nums[::2])      # [0, 2, 4, 6, 8] -- every 2nd element
5 print(nums[::-1])     # reversed list
6 print(nums[-3:])      # last 3 elements
```

Tuples are immutable and hashable, used for fixed-size records such as image dimensions (width, height = (1920, 1080)) and as dictionary keys. **Dictionaries** store key–value pairs with $O(1)$ average lookup; `.get(key, default)` avoids `KeyError` when a key may not exist, and `.update()` merges another dict in (overwriting duplicate keys). **Sets** provide $O(1)$ membership testing and standard set-algebra operations:

Listing 2: Dict and set idioms

```
1 student = {"name": "Tanmay", "scores": {"week1": 90, "week2": 85}}
2 print(student.get("gpa", "N/A"))      # N/A -- no KeyError
3 student.update({"mentor": "Sandipan"}) # add key
4 print(student["scores"]["week1"])      # 90 -- nested access
5
6 a, b = {1, 2, 3, 4}, {3, 4, 5, 6}
7 print(a.union(b))                     # {1,2,3,4,5,6}
8 print(a.intersection(b))              # {3,4}
9 print(a.difference(b))                 # {1,2}
```

`enumerate` pairs each element with its index; `zip` walks multiple iterables in parallel. Both avoid manual index bookkeeping. **List comprehensions** build new lists in a single expression, generally more concise and faster than an equivalent explicit loop:

Listing 3: `enumerate`, `zip`, and list comprehensions

```

1 for i, color in enumerate(["red", "green", "blue"]):
2     print(i, color)                                # 0 red / 1 green / 2 blue
3
4 names = ["Tanmay", "Sandipan"]
5 roles = ["Trainee", "Mentor"]
6 for name, role in zip(names, roles):
7     print(f"{name}: {role}")
8
9 squares = [x**2 for x in range(10)]
10 normalized = [p / 255.0 for p in [0, 64, 128, 255]]
11 flat = [val for row in [[1,2,3],[4,5,6]] for val in row] # flatten

```

2.2 NumPy

NumPy's `ndarray` is a fixed-type, n -dimensional grid of numbers — the data structure every OpenCV image actually is. An 8-bit color image is a (`height`, `width`, 3) array of `uint8` values.

Listing 4: Creating, inspecting, and slicing arrays

```

1 import numpy as np
2
3 b = np.array([[1, 2, 3], [4, 5, 6]])
4 print(b.shape, b.dtype, b.ndim)    # (2, 3) int64 2
5
6 zeros = np.zeros((3, 3))           # 3x3 array of 0.0
7 ones = np.ones((2, 4))             # 2x4 array of 1.0
8 seq = np.arange(0, 10, 2)          # [0, 2, 4, 6, 8]
9
10 img = np.zeros((4, 4))
11 img[1, 2] = 9                      # row 1, column 2
12 print(img[0:2, 0:2])               # top-left 2x2 block
13 print(img[:, 1])                   # entire column 1
14 flat = img.reshape(16)             # flatten to 1D

```

Listing 5: Vectorized operations and broadcasting

```

1 a = np.array([1, 2, 3])
2 b = np.array([10, 20, 30])
3
4 print(a + b)                        # [11, 22, 33] -- element-wise, no loop needed
5 print(a * 2)                       # [2, 4, 6] -- scalar broadcast
6 print(b > 15)                      # [False, True, True] -- boolean mask

```

```

7 print(b[b > 15])          # [20, 30]      -- masked selection
8 print(a.sum(), a.mean())  # 6  2.0

```

Broadcasting automatically stretches a smaller array (such as a scalar) across the shape of a larger one. *Boolean masking* selects elements satisfying a condition. Together, these make operations such as `img > 127` (thresholding) or `img / 255.0` (normalizing) run as a single vectorized expression rather than a Python-level loop — critical performance when an image has hundreds of thousands of pixels.

3 OpenCV Fundamentals

Before building the colored object tracker, a set of core OpenCV operations was worked through covering media I/O, image processing, drawing, geometric transforms, and contour-based shape analysis.

3.1 Reading Images, Video, and Webcam Streams

OpenCV treats all three input sources through the same interface. A static image is loaded with `cv2.imread` and displayed with `cv2.imshow`. Video and webcam feeds are continuous streams of frames handled by opening a `VideoCapture` object and reading frames in a loop:

Listing 6: Live webcam feed

```

1 import cv2
2 cap = cv2.VideoCapture(0)          # 0 = default webcam
3 cap.set(3, 640); cap.set(4, 480)  # property IDs 3=width, 4=height
4 while True:
5     success, img = cap.read()
6     cv2.imshow("Result", img)
7     if cv2.waitKey(1) & 0xFF == ord('q'):
8         break
9 cap.release()
10 cv2.destroyAllWindows()

```

The only practical difference between a video file and a webcam is the argument to `VideoCapture`: a file path string versus a device index (0 for the default camera). The numeric codes 3 and 4 are equivalent to the named constants `cv2.CAP_PROP_FRAME_WIDTH` and `cv2.CAP_PROP_FRAME_HEIGHT`.

3.2 Basic Image Processing Pipeline

A typical preprocessing pipeline converts a color image to grayscale, smooths it, finds edges, then cleans those edges morphologically:

Listing 7: Grayscale, blur, Canny edges, and morphology

```

1 import cv2, numpy as np
2 kernel = np.ones((5, 5), np.uint8)

```

```

3 img      = cv2.imread("lena.png")
4
5 imgGray   = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
6 imgBlur   = cv2.GaussianBlur(imgGray, (7, 7), 0)
7 imgCanny  = cv2.Canny(imgBlur, 100, 200)
8 imgDilated = cv2.dilate(imgCanny, kernel, iterations=3)
9 imgEroded  = cv2.erode(imgDilated, kernel, iterations=1)

```

`cvtColor` converts between color spaces; here BGR to single-channel grayscale since edge detection does not require color. `GaussianBlur` suppresses high-frequency noise before edge detection, which otherwise produces a very noisy edge map. `Canny` applies a two-threshold (hysteresis) edge detector: gradients above the upper threshold are immediately accepted as edges, below the lower threshold are rejected, and those in between are kept only if connected to a strong edge. `dilate` grows white regions using the 5×5 structuring element, closing small gaps in a broken edge map; `erode` reverses this, shrinking white regions to remove noise without losing the main structure.

3.3 Resizing, Cropping, and Drawing

Since an OpenCV image is a NumPy array, cropping is just array slicing. A common pitfall: `img.shape` returns (**height, width, channels**), while `cv2.resize` expects (**width, height**) — the two are transposed relative to each other. Cropping slices the *row* range (vertical / *y*) first, then the *column* range (horizontal / *x*):

Listing 8: Resize, crop, and draw on a blank canvas

```

1 imgResize = cv2.resize(img, (1000, 1000)) # (width, height)
2 imgCropped = img[300:540, 430:480]        # [rows, then cols]
3
4 canvas = np.zeros((512, 512, 3), np.uint8)
5 cv2.line(canvas, (0,0), (512,512), (0,255,0), 2)
6 cv2.rectangle(canvas, (350,100), (450,200), (0,0,255), cv2.FILLED)
7 cv2.circle(canvas, (150,400), 50, (255,0,0), 3)
8 cv2.putText(canvas, "Shapes", (75,50),
9             cv2.FONT_HERSHEY_COMPLEX, 1, (0,150,0), 1)

```

Colors are BGR tuples (not RGB). A negative thickness or `cv2.FILLED` fills the shape solid rather than drawing only its outline.

3.4 Displaying Multiple Images (stackImages)

Calling `cv2.imshow` repeatedly opens one window per image, which is inconvenient when comparing processing stages. A `stackImages` helper resizes every image to a common size, converts any single-channel (grayscale or binary) image to 3-channel BGR so it can be joined with color images, and combines everything into a single grid using `np.hstack` (horizontal) and `np.vstack` (vertical):

Listing 9: Core logic of the image-stacking utility

```

1 def stackImages(imgArray, scale):
2     rows, cols = len(imgArray), len(imgArray[0])
3     width  = imgArray[0][0].shape[1]
4     height = imgArray[0][0].shape[0]
5     for x in range(rows):
6         for y in range(cols):
7             imgArray[x][y] = cv2.resize(imgArray[x][y],
8                                         (width, height), None, scale, scale)
9             if len(imgArray[x][y].shape) == 2:  # single-channel
10                imgArray[x][y] = cv2.cvtColor(
11                    imgArray[x][y], cv2.COLOR_GRAY2BGR)
12     hor = [np.hstack(row) for row in imgArray]
13     return np.vstack(hor)

```

Passing a nested list such as `[[img, imgGray], [imgCanny, imgDilated]]` produces a 2×2 grid in a single window. This was used throughout the color tracker tuning process, where seeing the original frame, mask, and detected result side by side made it immediately obvious when a threshold was wrong.

3.5 Perspective Transformation

A photograph of a flat object (such as a playing card) taken at an angle can be “unwarped” into a clean top-down view. Four source points (corners of the object in the original image) are mapped onto four destination points (corners of a clean rectangle):

Listing 10: Perspective warp from tilted to flat

```

1 pts1 = np.float32([[111,219],[287,188],[154,482],[352,440]])
2 pts2 = np.float32([[0,0],[250,0],[0,350],[250,350]])
3 matrix = cv2.getPerspectiveTransform(pts1, pts2)
4 imgOutput = cv2.warpPerspective(img, matrix, (250, 350))

```

`getPerspectiveTransform` solves for the 3×3 homography matrix that maps `pts1` onto `pts2`; `warpPerspective` applies it to every pixel. For interactive use, a mouse callback collects the four corner points by clicking on the image, replacing the hardcoded `pts1`:

Listing 11: Collecting corner points interactively via mouse

```

1 clicks = np.zeros((4, 2), int)
2 counter = 0
3 def mousePoints(event, x, y, flags, params):
4     global counter
5     if event == cv2.EVENT_LBUTTONDOWN:
6         clicks[counter] = x, y
7         counter += 1
8 cv2.setMouseCallback("Original Image", mousePoints)

```

Once four clicks are registered, `pts1` is built from `clicks` and the same transform pipeline runs as before.

3.6 HSV Color Space and Live Color Masking

Thresholding directly in BGR is brittle because a single perceptual color is spread across all three channels in a way that shifts with lighting. Converting to HSV (Hue, Saturation, Value) separates *which color* a pixel is (the hue) from how saturated and how bright it is, making color-based segmentation far more robust:

Listing 12: Live HSV threshold tuning with trackbars

```
1 def empty(a): pass
2 cv2.createTrackbar("HUE Min", "HSV", 0, 179, empty)
3 cv2.createTrackbar("HUE Max", "HSV", 179, 179, empty)
4 # similarly for SAT Min/Max and VALUE Min/Max ...
5
6 imgHsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
7 lower = np.array([h_min, s_min, v_min])
8 upper = np.array([h_max, s_max, v_max])
9 mask = cv2.inRange(imgHsv, lower, upper)
10 result = cv2.bitwise_and(img, img, mask=mask)
```

`cv2.inRange` produces a binary mask: a pixel is white if every one of its H, S, and V values falls inside the given bounds, and black otherwise. `bitwise_and` then uses that mask to keep only matching pixels of the original frame, isolating the target color. The trackbars expose all six bounds as live sliders, so dragging one takes effect immediately rather than requiring a script restart. This is precisely why tuning the tracker revealed that the target shirt's actual hue fell in the magenta range (~ 148 – 170) rather than the conventionally expected red range (0 – 10).

3.7 Contour Detection and Shape Analysis

The final piece of the toolkit identifies discrete objects in a frame. The pipeline blurs, grayscales, and Canny-detects the frame, dilates once to close gaps, then finds and annotates contours:

Listing 13: Detecting and classifying contours

```
1 def getContours(img, imgContour):
2     contours, _ = cv2.findContours(img, cv2.RETR_EXTERNAL,
3                                     cv2.CHAIN_APPROX_NONE)
4     for cnt in contours:
5         area = cv2.contourArea(cnt)
6         if area > areaMin:
7             peri = cv2.arcLength(cnt, True)
8             approx = cv2.approxPolyDP(cnt, 0.02*peri, True)
9             x, y, w, h = cv2.boundingRect(approx)
```

```

10         cv2.drawContours(imgContour, cnt, -1, (255,0,255), 7)
11         cv2.rectangle(imgContour, (x,y), (x+w,y+h), (0,255,0),
12                        5)
13         cv2.putText(imgContour,
14                     f"Points: {len(approx)} Area: {int(area)}",
15                     (x+w+5, y+20), cv2.FONT_HERSHEY_COMPLEX,
16                     0.7, (0,255,0), 2)

```

`findContours` with `RETR_EXTERNAL` retrieves only the outermost boundary of each connected white region. `contourArea` compared against a trackbar-set minimum discards small noise blobs. `approxPolyDP` simplifies the contour to corner vertices with tolerance $\varepsilon = 0.02 \times$ perimeter, proportional to the shape's own size — the resulting vertex count distinguishes a triangle from a rectangle from a more complex polygon. `boundingRect` gives the axis-aligned box drawn in green, annotated with the vertex count and measured area.

4 Midterm Deliverable: Real-Time Colored Object Tracker

The Week 3 mini-project: a script that opens the webcam, lets the user tune an HSV color range live with trackbars, masks everything except that color, and draws a bounding box with the detected area around whatever remains. The code is walked through block by block below.

4.1 Webcam Setup and Trackbar Windows

Listing 14: Opening the webcam and creating trackbar windows

```

1 import cv2, numpy as np
2
3 cap = cv2.VideoCapture(0)
4 cap.set(3, 640); cap.set(4, 480)
5
6 def empty(a): pass
7
8 cv2.namedWindow("HSV"); cv2.resizeWindow("HSV", 640, 240)
9 cv2.createTrackbar("HUE Min", "HSV", 0, 179, empty)
10 cv2.createTrackbar("HUE Max", "HSV", 10, 179, empty)
11 cv2.createTrackbar("SAT Min", "HSV", 30, 255, empty)
12 cv2.createTrackbar("SAT Max", "HSV", 255, 255, empty)
13 cv2.createTrackbar("VALUE Min", "HSV", 100, 255, empty)
14 cv2.createTrackbar("VALUE Max", "HSV", 255, 255, empty)
15
16 cv2.namedWindow("Parameters"); cv2.resizeWindow("Parameters", 640,
17          100)
18 cv2.createTrackbar("Area", "Parameters", 1000, 30000, empty)

```

`empty()` is a no-op callback — `createTrackbar` requires one, but all reading is done manually via `getTrackbarPos`. Two windows are created: one with six HSV sliders and one with a single area threshold. The starting values (HUE Min 0, HUE Max 10) are the textbook “red” assumption — which the tuning session proved wrong.

4.2 Finding and Drawing the Object

Listing 15: Contour detection and bounding box

```

1 def getContours(img, imgContour):
2     contours, _ = cv2.findContours(img, cv2.RETR_EXTERNAL,
3                                     cv2.CHAIN_APPROX_NONE)
4     areaMin = cv2.getTrackbarPos("Area", "Parameters")
5     for cnt in contours:
6         area = cv2.contourArea(cnt)
7         if area > areaMin:
8             peri = cv2.arcLength(cnt, True)
9             approx = cv2.approxPolyDP(cnt, 0.02*peri, True)
10            x, y, w, h = cv2.boundingRect(approx)
11            cv2.drawContours(imgContour, cnt, -1, (255,0,255), 7)
12            cv2.rectangle(imgContour, (x,y), (x+w,y+h), (0,255,0),
13                           5)
14            cv2.putText(imgContour, "Area: "+str(int(area)),
15                        (x, y-10), cv2.FONT_HERSHEY_COMPLEX,
16                        0.7, (0,255,0), 2)

```

The area trackbar filters stray noise blobs below the threshold, so only the main object survives. `approxPolyDP` with $0.02 \times$ perimeter works regardless of object size on screen. The green bounding box and “Area: ...” label are visible in both screenshots that follow.

4.3 Main Loop

Listing 16: Per-frame processing: HSV mask, contour, display

```

1 while True:
2     success, img = cap.read()
3     if not success: break
4
5     imgHsv = cv2.cvtColor(img, cv2.COLOR_BGR2HSV)
6     h_min = cv2.getTrackbarPos("HUE Min", "HSV")
7     h_max = cv2.getTrackbarPos("HUE Max", "HSV")
8     s_min = cv2.getTrackbarPos("SAT Min", "HSV")
9     s_max = cv2.getTrackbarPos("SAT Max", "HSV")
10    v_min = cv2.getTrackbarPos("VALUE Min", "HSV")
11    v_max = cv2.getTrackbarPos("VALUE Max", "HSV")
12
13    mask = cv2.inRange(imgHsv,
14                        np.array([h_min, s_min, v_min]),

```

```

15         np.array([h_max, s_max, v_max]))
16     imgContour = img.copy()
17     getContours(mask, imgContour)
18
19     maskBGR = cv2.cvtColor(mask, cv2.COLOR_GRAY2BGR)
20     imgStack = stackImages(0.7, ([img, maskBGR],
21                                 [imgContour, imgContour]))
22     cv2.imshow("Result", imgStack)
23     if cv2.waitKey(1) & 0xFF == ord('q'): break
24
25 cap.release(); cv2.destroyAllWindows()

```

Key design decisions: trackbar values are read *fresh on every frame*, so dragging a slider takes effect immediately. `img.copy()` ensures the original frame is not mutated before drawing. Converting the single-channel mask to BGR before stacking is mandatory — `np.hstack` cannot join arrays with different channel counts. `cap.release()` and `destroyAllWindows()` prevent the webcam from staying locked after exit.

4.4 Tuning Session and Final Result

Figure 1 is a screenshot from partway through the tuning session. The hue sliders settled at roughly 139–171 to isolate the pink object — quite different from the initial 0–10 (“red”) assumption. Without live trackbars this gap would have been very easy to miss: the object’s true hue sits in the magenta band, detectable only by exploring the color wheel interactively.

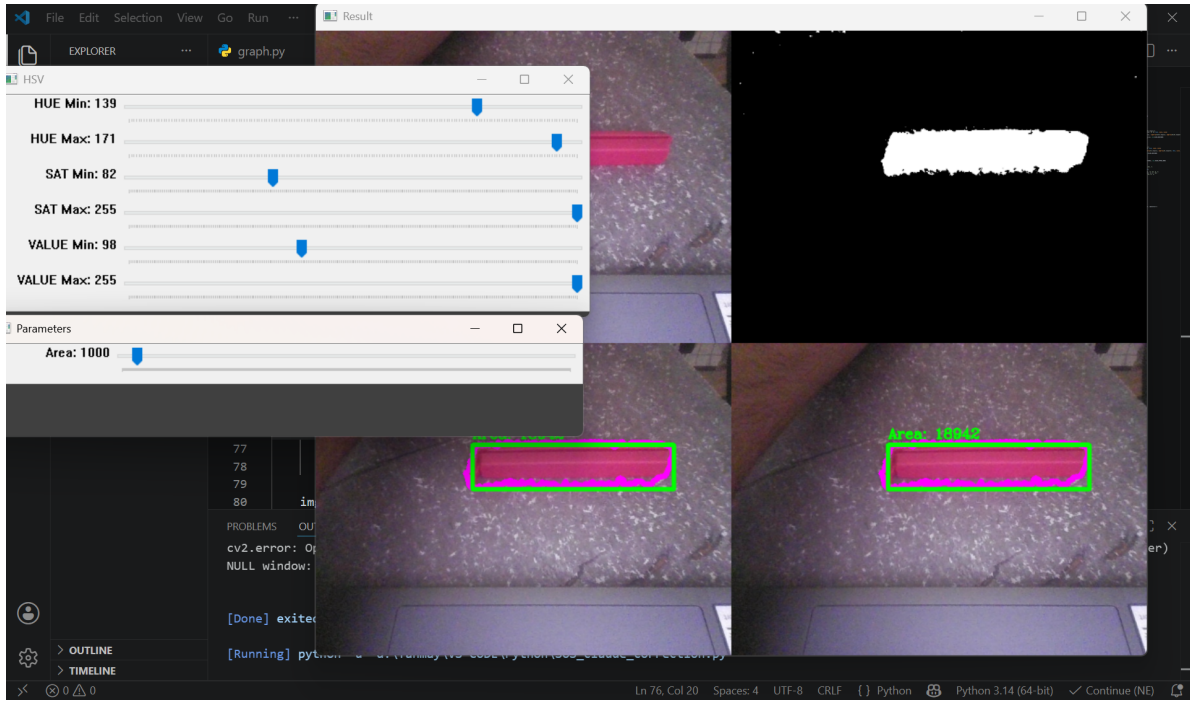


Figure 1: Live HSV tuning: Hue 139–171, Saturation 82–255, Value 98–255, area threshold 1000. The mask is still slightly noisy at this stage of tuning.

Figure 2 is the clean final run: original frame (top-left), binary mask (top-right), and the bounding box with measured area 22432 on the bottom row. The two bottom panels are identical because `imgContours` is passed twice to keep the 2×2 grid visually balanced.

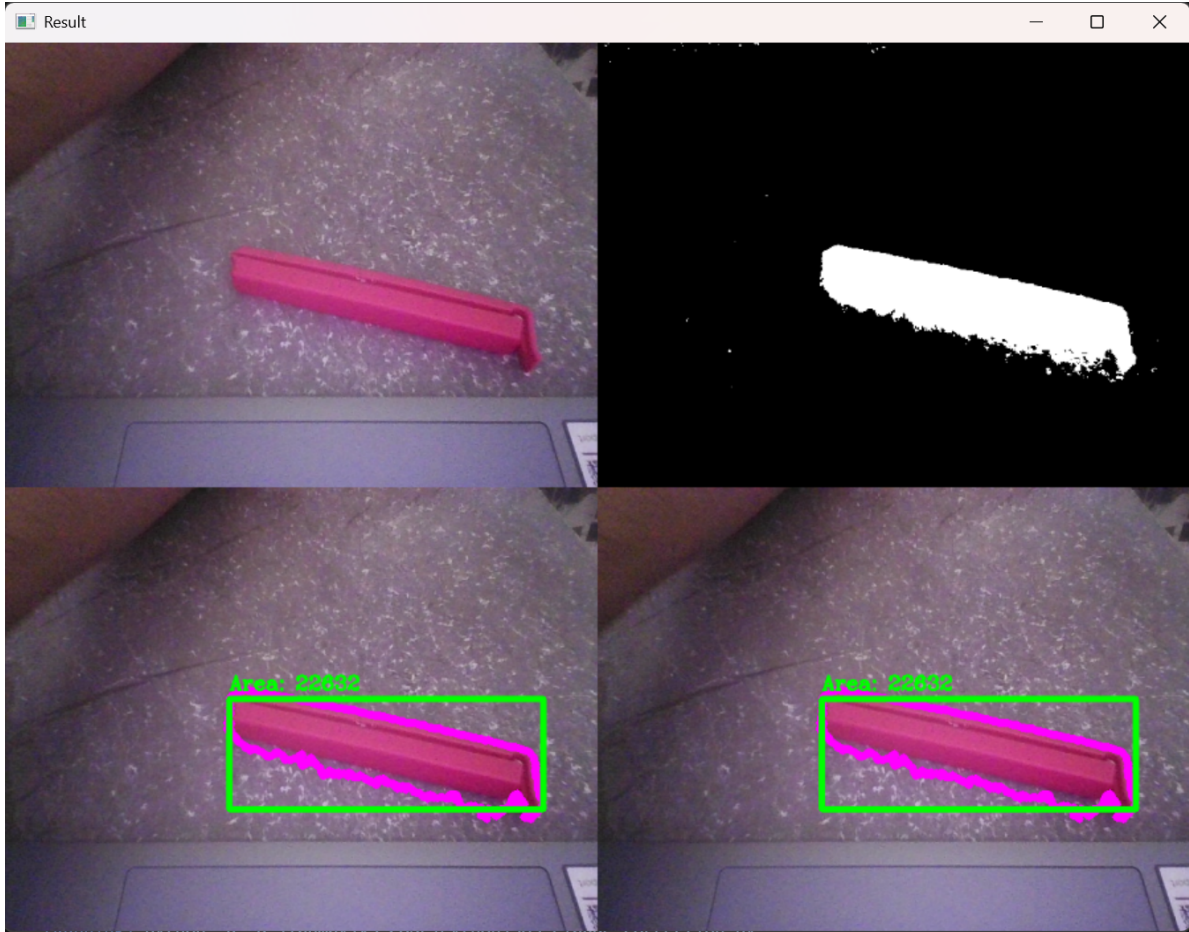


Figure 2: Final tracker output: clean HSV mask, accurate bounding box, measured contour area of 22432.

5 Neural Network Foundations

While the formal syllabus centers on classical computer vision, the 3Blue1Brown “Neural Networks” series was used as a self-directed bridge into deep learning — first the mechanics of a feedforward network trained by gradient descent and backpropagation, then how those ideas scale into the transformer architecture, and finally how embeddings and denoising are used to generate images.

5.1 The Neuron and Activation

A neuron holds a single number in $[0, 1]$ called its *activation*. For a 28×28 greyscale image (MNIST handwritten digits), each of the 784 pixels becomes one input neuron’s activation, with 0 corresponding to black and 1 to white. The output layer for a 10-class classifier has 10 neurons; the one with the highest activation after a forward pass is taken as the network’s prediction.

Each neuron in a hidden layer computes a weighted sum of all activations in the previous

layer, adds a learned bias, and squashes the result through the logistic sigmoid to produce its own activation:

$$\text{activation} = \sigma\left(\sum_i w_i a_i + b\right), \quad \sigma(x) = \frac{1}{1 + e^{-x}}.$$

The bias b shifts the effective threshold: the neuron only fires meaningfully once the weighted sum exceeds some learned value. In matrix form the full layer transition is

$$a^{(1)} = \sigma(Wa^{(0)} + b), \quad W = \begin{bmatrix} w_{0,0} & w_{0,1} & \cdots & w_{0,n} \\ w_{1,0} & w_{1,1} & \cdots & w_{1,n} \\ \vdots & & \ddots & \\ w_{k,0} & w_{k,1} & \cdots & w_{k,n} \end{bmatrix},$$

where W is the weight matrix, σ is applied element-wise, and each row of W represents one neuron in the next layer querying every neuron in the previous layer. For a 784–16–16–10 network: $784 \times 16 + 16 \times 16 + 16 \times 10 = 12,960$ weights plus $16 + 16 + 10 = 42$ biases, totalling **13,002** learnable parameters.

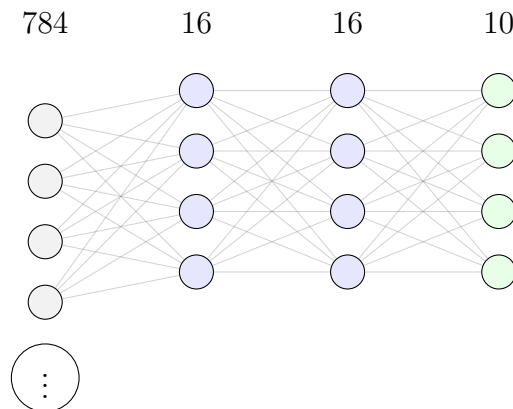


Figure 3: A 784–16–16–10 feedforward network for MNIST digit classification. Only a representative subset of nodes and edges is drawn.

5.2 Cost Function and Gradient Descent

The squared-error cost for one training example compares the output activations $a^{(L)}$ to the one-hot target y :

$$C_0 = \sum_j \left(a_j^{(L)} - y_j\right)^2.$$

The average over the training set,

$$C = \frac{1}{n} \sum_x C_x,$$

is the single scalar measuring overall performance. Training means minimizing C over all 13,002 parameters. Gradient descent does this iteratively: compute the gradient, take a small step in the direction of $-\nabla C$, and repeat:

$$\vec{w} \leftarrow \vec{w} - \eta \nabla C(\vec{w}), \quad \nabla C = \begin{bmatrix} \partial C / \partial w_1 \\ \vdots \\ \partial C / \partial b_k \end{bmatrix},$$

where η is the learning rate. This is a 13,002-dimensional analogue of always walking downhill; the gradient tells us which combination of nudges to all parameters decreases the cost fastest from the current point.

5.3 Backpropagation

Backpropagation computes ∇C efficiently by propagating the error backward from the output layer using the chain rule. For a single output neuron, define

$$z^{(L)} = w^{(L)} a^{(L-1)} + b^{(L)}, \quad a^{(L)} = \sigma(z^{(L)}).$$

The chain rule gives the sensitivity of the cost to the weight $w^{(L)}$:

$$\frac{\partial C_0}{\partial w^{(L)}} = \underbrace{\frac{\partial z^{(L)}}{\partial w^{(L)}}}_{a^{(L-1)}} \cdot \underbrace{\frac{\partial a^{(L)}}{\partial z^{(L)}}}_{\sigma'(z^{(L)})} \cdot \underbrace{\frac{\partial C_0}{\partial a^{(L)}}}_{2(a^{(L)} - y)} = a^{(L-1)} \sigma'(z^{(L)}) 2(a^{(L)} - y).$$

The sensitivity of the cost to the *previous* layer's activation is

$$\frac{\partial C_0}{\partial a^{(L-1)}} = w^{(L)} \sigma'(z^{(L)}) 2(a^{(L)} - y),$$

which is the key: this passes the error one layer further back. With multiple neurons per layer (indices j, k) this generalizes to

$$\frac{\partial C_0}{\partial a_k^{(L-1)}} = \sum_j w_{jk}^{(L)} \sigma'(z_j^{(L)}) 2(a_j^{(L)} - y_j).$$

The influence of neuron k on the cost is a sum over every neuron in the next layer it feeds into — this summation structure is what allows the error to flow through the entire network. Averaging $\partial C_x / \partial w$ over a mini-batch gives the practical gradient used at each step.

Stochastic Gradient Descent (SGD) divides the training set into small mini-batches and takes one gradient step per mini-batch. This trades a small amount of randomness in the descent path for a large reduction in compute per step — analogous to a drunk man taking many small, slightly noisy steps downhill rather than computing the exact steepest direction at every point, yet still converging to the bottom faster overall.

5.4 Neural Network Variants

The plain feedforward MLP is the conceptual foundation for more specialized architectures. **Convolutional Neural Networks (CNNs)** exploit spatial locality: a convolutional filter (e.g. 3×3) slides across an image sharing weights at every position, so a filter with 64 channels has only 576 parameters regardless of image size. Successive pooling layers progressively reduce spatial resolution, building up abstract feature representations. This makes CNNs dramatically more parameter-efficient for image tasks than a fully-connected MLP would be. **Long Short-Term Memory networks (LSTMs)** are designed for sequential data such as speech and time series, using three gating mechanisms (input gate, forget gate, output gate) to selectively retain or discard information across time steps — addressing the vanishing gradient problem that causes plain recurrent networks to “forget” long-range dependencies. Understanding how weights, activations, cost, and backpropagation work in the MLP transfers directly to both: CNNs and LSTMs add architectural structure on top of the same underlying principles.

6 Large Language Models and the Transformer Architecture

6.1 What an LLM Is

A large language model is a mathematical function that predicts the most likely next token for any given piece of text. Modern LLMs have hundreds of billions of parameters, refined from random initialization by exposure to vast text corpora and a final RLHF (Reinforcement Learning from Human Feedback) alignment stage. Training at this scale relies entirely on GPUs, which exploit the highly parallel structure of the underlying matrix operations — naively running on a single CPU performing 10^9 operations per second would take on the order of 100 million years for a model the size of GPT-3.

6.2 Data Flow Through a Transformer

Transformers process an entire input sequence in parallel rather than token-by-token. Two repeating building blocks — an **attention** block (vectors exchange contextual information) and an **MLP** block (each vector is refined independently) — alternate many times before a final output is produced:

1. **Tokenization and embedding.** Input tokens are mapped to high-dimensional vectors via the embedding matrix W_E .
2. **Attention.** Vectors are updated by aggregating information from contextually relevant tokens in the sequence.
3. **MLP.** Each context-aware vector is passed independently through a feed-forward layer.
4. **Unembedding.** After the Attention/MLP pair repeats (96 times in GPT-3), the final vector is projected back to a probability distribution over the vocabulary.

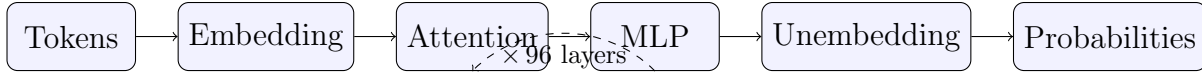


Figure 4: Data flow through a transformer. In GPT-3, the Attention/MLP pair repeats 96 times before unembedding.

6.3 Embeddings

The embedding matrix W_E converts each token into a high-dimensional vector. The model learns W_E such that directions in the resulting space acquire meaningful semantic structure — gender, plurality, and historical association are examples of directions that emerge naturally from training, so that semantically similar words land close together. A frequently cited illustration:

$$E(\text{queen}) \approx E(\text{king}) + E(\text{women}) - E(\text{men}).$$

In GPT-3 the embedding space is 12,288-dimensional over a vocabulary of 50,257 tokens, giving 617,558,016 parameters in W_E alone. The context size is 2048 tokens, meaning the model works with a $2048 \times 12,288$ array of vectors at any one time.

6.4 The Attention Mechanism

Within one attention head, every token’s embedding $\vec{E}_i$ is projected through three learned matrices into a Query, Key, and Value:

$$\vec{Q}_i = W_Q \vec{E}_i, \quad \vec{K}_i = W_K \vec{E}_i, \quad \vec{V}_i = W_V \vec{E}_i.$$

The Query asks “what kind of information am I looking for?”; the Key answers “what kind of information do I contain?”; the Value carries the information to be added if a match is found. Relevance between every pair of tokens is computed as the dot product $\vec{K}_i \cdot \vec{Q}_j$, forming the *attention pattern*. Future tokens are masked (set to $-\infty$) before softmax so the model cannot look ahead. The compact formula is:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{K^\top Q}{\sqrt{d_k}}\right) V,$$

where d_k is the key/query dimension (used to keep dot products numerically stable). GPT-3 runs 96 attention heads in parallel, each learning a different type of relationship (grammatical structure, entity associations, sentiment, etc.), and sums all their updates back into the embedding. *Cross-attention* is a variant where the query and key projections act on two distinct sequences (e.g. source and target language in translation), rather than a single sequence attending to itself (self-attention).

6.5 MLP Blocks and Softmax

The MLP block applies to each token vector independently via three stages: (1) an up-projection W_{up} that maps each input to a high-dimensional intermediate space; (2) a nonlinear activation (ReLU or GELU) that acts as a gating mechanism, firing only when the input

strongly matches a learned pattern; (3) a down-projection W_{down} that writes the result back into the residual stream. The MLP blocks are believed to store most of the model’s factual knowledge — roughly two-thirds of GPT-3’s total parameters reside here.

Softmax converts raw output logits to a valid probability distribution, with a temperature parameter T controlling output diversity:

$$p_i = \frac{e^{x_i/T}}{\sum_k e^{x_k/T}}.$$

Low T concentrates probability on the single most likely token; high T flattens the distribution, allowing more varied (and sometimes less coherent) outputs.

6.6 GPT-3 Parameter Breakdown

Component	Parameters
Embedding matrix	617,558,016
Attention (Q, K, V, O; 96 heads $\times$ 96 layers)	57,982,058,496
MLP (up- and down-projection; 96 layers)	115,964,116,992
Unembedding matrix	617,558,016
Total	175,181,291,520

Table 1: Parameter breakdown of GPT-3, totalling approximately 175 billion parameters.

6.7 Superposition

An n -dimensional vector space admits at most n mutually perpendicular vectors, but far more that are *nearly* perpendicular (in the 89° – 91° range), and the number of such near-orthogonal directions grows extremely quickly with dimension. This phenomenon — *superposition* — lets an LLM represent many more distinct concepts than it has literal dimensions. It is closely related to the Johnson–Lindenstrauss lemma in dimensionality reduction.

7 Generative Models for Images

7.1 CLIP: Bridging Text and Vision

Modern generative image models rely on a shared high-dimensional latent space connecting language and vision. CLIP (Contrastive Language–Image Pre-training) achieves this through dual encoders: an image encoder and a text encoder that independently map their inputs into the same 512-dimensional vector space. Training is *contrastive*: for a batch of image–caption pairs, CLIP maximizes the cosine similarity

$$\cos \theta = \frac{v_1 \cdot v_2}{\|v_1\| \|v_2\|}$$

between matching pairs while minimizing it for all non-matching pairs. This yields an embedding space where visual and linguistic concepts are geometrically aligned, enabling vector arithmetic (e.g. adding a concept direction to an image embedding shifts it semantically) and making it possible to rank images against text prompts without task-specific fine-tuning.

7.2 Diffusion Models

Diffusion models generate images by learning to reverse a structured noise process modelled on Brownian motion. *Forward diffusion* incrementally adds Gaussian noise to a clean image x_0 across T steps until it becomes indistinguishable from pure Gaussian noise x_T . *Reverse diffusion* is the actual learning task: the model is trained to predict the total noise that was added to a noisy image, rather than reversing one step at a time — this reduces variance and makes training more stable. The full process is formalized as a stochastic differential equation (SDE):

$$dx = [f(x, t) - g(t)^2 \nabla_x \log p_t(x)] dt + g(t) d\bar{w},$$

where $f(x, t)$ is the drift coefficient (deterministic part), $g(t)$ the diffusion coefficient (noise scale), $\nabla_x \log p_t(x)$ is the *score function* pointing toward higher-probability, less noisy data, and $d\bar{w}$ is a standard Wiener process. The simplified DDPM (Denoising Diffusion Probabilistic Models) update rule is:

$$x_{t-1} = x_t + a_t \varepsilon_\theta(x_t, \sigma_t) + \eta w_t,$$

where ε_θ is the model’s predicted noise and ηw_t is freshly injected Gaussian noise.

7.3 Deterministic Sampling: DDIM

Removing the random component from the SDE converts it to an ordinary differential equation (ODE), preserving output quality while making the generation process fully deterministic. DDIM (Denoising Diffusion Implicit Models) builds on this by removing randomness and adjusting the step scaling, enabling high-quality image synthesis in 20–50 steps rather than the ~ 1000 required by the stochastic process. This determinism also means that the same starting noise vector always produces the same image, which is useful for controlled image editing and interpolation.

8 Convolutional Neural Networks

Week 5 extended the neural network foundations of Section 5 into Convolutional Neural Networks, following Stanford CS231n (Lecture 5, Fei-Fei Li et al.) and Andrew Ng’s Deep Learning Specialization Course 1. This section documents the score-function/loss-function view of a classifier, then the layer types and spatial-arrangement mechanics that make ConvNets efficient for images.

8.1 From Linear Classifiers to Neural Networks

An image classifier is a function $f(x, W)$ mapping raw pixels to class scores. For a CIFAR-10 image (an array of $32 \times 32 \times 3 = 3072$ numbers), a linear classifier is simply

$$f(x, W) = Wx + b,$$

producing 10 numbers, one score per class, where W is the matrix of learned parameters (weights). A single linear layer is not very powerful, since it can only separate classes with a hyperplane in pixel space. Stacking a nonlinearity in between two linear layers gives a genuine neural network:

$$\text{Linear: } f = Wx \quad \longrightarrow \quad \text{2-layer NN: } f = W_2 \max(0, W_1 x).$$

For CIFAR-10 this looks like $3072 \rightarrow 100 \rightarrow 10$: the 3072-dimensional input is projected down to a 100-dimensional hidden layer h , passed through a ReLU ($\max(0, x)$), and then projected to 10 class scores.

8.2 Loss Functions

Given a dataset of pairs (x_i, y_i) and a score function $s = f(x_i; W) = Wx_i$, training requires a scalar measure of how wrong the current scores are.

Softmax loss interprets scores as unnormalized log-probabilities. The loss for one example is the negative log-probability assigned to the correct class:

$$L_i = -\log \left(\frac{e^{s_{y_i}}}{\sum_j e^{s_j}} \right).$$

Hinge loss (multiclass SVM loss) instead penalizes any incorrect-class score that comes within a margin of the correct-class score:

$$L_i = \sum_{j \neq y_i} \max(0, s_j - s_{y_i} + 1).$$

Either per-example loss is averaged over the dataset and combined with a regularization term $R(W)$ that penalizes large weights (discouraging overfitting) to give the **full loss**:

$$L = \frac{1}{N} \sum_{i=1}^N L_i + R(W).$$

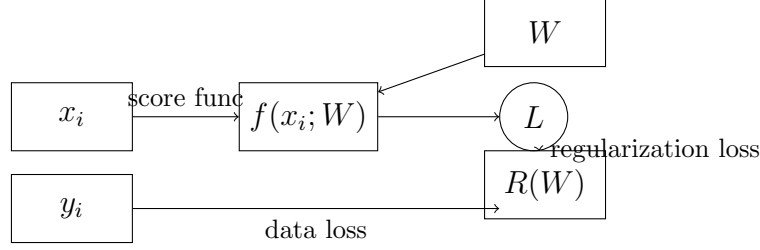


Figure 5: Computation graph of the full loss: a score function combines input and weights into a data loss, while a separate regularization term penalizes the weights directly, and the two sum into the total loss L .

8.3 CNN Architecture and Motivation

ConvNet architectures make the explicit assumption that the inputs are images, which allows the network to encode certain properties into its structure. These assumptions make the forward function more efficient to implement and vastly reduce the number of parameters compared to a fully-connected network of the same depth.

A ConvNet arranges its neurons in three dimensions: **width, height, and depth**. For CIFAR-10, the input volume is $32 \times 32 \times 3$ (width, height, 3 color channels), and the final output layer has dimensions $1 \times 1 \times 10$ — one score per class. A ConvNet is built from layers, each of which transforms an input 3D volume into an output 3D volume through a differentiable function, some of which have learnable parameters and some of which do not.

A typical architecture stacks four layer types in the pattern **INPUT – CONV – RELU – POOL – FC**:

INPUT holds the raw pixels, e.g. $[32 \times 32 \times 3]$ for CIFAR-10.

CONV computes the output of neurons connected to local regions of the input, each computing a dot product between its weights and the small region it is connected to. With 12 filters this may produce a volume such as $[32 \times 32 \times 12]$.

RELU applies an elementwise activation such as $\max(0, x)$, thresholding at zero. This leaves the volume size unchanged: $[32 \times 32 \times 12] \rightarrow [32 \times 32 \times 12]$.

POOL performs a downsampling operation along the spatial dimensions (width, height), producing a volume such as $[16 \times 16 \times 12]$.

FC (fully-connected) computes the class scores, resulting in a volume of size $[1 \times 1 \times 10]$, where each of the 10 numbers corresponds to a class score. As in an ordinary neural network, every neuron here is connected to all numbers in the previous volume.

Only the CONV and FC layers have parameters that are trained via gradient descent; RELU and POOL are fixed functions.

8.4 The Convolutional Layer

Every filter is small spatially (along width and height), but extends through the *full depth* of the input volume. A typical filter on the first layer of a ConvNet might have size $5 \times 5 \times 3$. During the forward pass, the filter slides (convolves) across the width and height of the input volume, producing a 2-dimensional **activation map** that gives the responses of that filter at every spatial position.

Local connectivity. Each neuron is connected to only a local region of the input volume, rather than the full input. The spatial extent of this connectivity is a hyperparameter called the neuron’s **receptive field** (equivalently, the filter size). For an input volume $[32 \times 32 \times 3]$ with filter size $[5 \times 5 \times 3]$, the total weight count per filter is $5 \times 5 \times 3 = 75$ weights plus one bias.

8.5 Spatial Arrangement

Three hyperparameters control the size of the output volume produced by a CONV layer:

Depth (K) corresponds to the number of filters used. Each filter learns to look for something different in the input (e.g. oriented edges, blobs of color). A set of neurons looking at the same local region of the input, one per filter, is referred to as a **depth column** (or fibre).

Stride (S) is the step size with which the filter slides across the input volume. A stride of 1 moves the filter one pixel at a time; a stride of 2 (or, uncommonly, 3) makes the filter jump 2 pixels at a time, producing spatially smaller output volumes.

Zero-padding (P) pads the input volume with zeros around its border. This is a convenient hyperparameter used to precisely control the spatial size of the output volume, most commonly to exactly preserve the input’s width and height.

Given input size W (width or height), the output size along that dimension is

$$\text{Output size} = \frac{W - F + 2P}{S} + 1,$$

where F is the receptive field (filter) size, S the stride, and P the amount of zero-padding.

Pooling layer. It is common to periodically insert a pooling layer in between successive CONV layers. Its function is to progressively reduce the spatial size of the representation, which reduces the amount of parameters and computation in the network and hence also helps control overfitting. For an input $W_1 \times H_1 \times D_1$ and hyperparameters F and S , the output volume $W_2 \times H_2 \times D_2$ is given by

$$W_2 = \frac{W_1 - F}{S} + 1, \quad H_2 = \frac{H_1 - F}{S} + 1, \quad D_2 = D_1.$$

Note that pooling operates independently on every depth slice, so the depth is left unchanged.

8.6 Real-World Case: AlexNet (2012)

AlexNet’s first convolutional layer illustrates the formulas above. With input $[227 \times 227 \times 3]$, filter size $F = 11$, stride $S = 4$, padding $P = 0$, and $K = 96$ filters:

$$\text{Output size} = \frac{227 - 11 + 2(0)}{4} + 1 = 55 \quad \implies \quad \text{Output volume} = [55 \times 55 \times 96],$$

which corresponds to $55 \times 55 \times 96 = 290,400$ neurons in the first layer alone, each with $11 \times 11 \times 3 = 363$ weights (plus a bias) — dramatically fewer parameters than a fully-connected layer of the same input size would require, which is precisely the parameter-sharing efficiency that motivates the convolutional architecture.

9 MediaPipe Hands: Real-Time Landmark Detection

Week 6 moved from classical CV and CNN theory into a pre-trained deep learning pipeline: Google’s MediaPipe Hands solution, which is the perception backbone of the end-term deliverable.

9.1 Pipeline Overview

MediaPipe Hands uses a two-stage pipeline. A **palm detector** first locates the hand(s) in the full frame, since hands are a comparatively small, high-contrast object that is easier to detect reliably than to track landmark-by-landmark from scratch on every frame. Once a palm bounding box is found, a **hand landmark model** operates on the cropped hand region and regresses precise **21 keypoints (landmarks)** per hand, each as (x, y, z) coordinates normalized to $[0, 1]$ relative to the image width and height, with z representing relative depth from the wrist.

Because the landmark model works on the cropped region, subsequent frames reuse the previous frame’s hand region as long as tracking confidence stays high, only re-invoking the (more expensive) palm detector when tracking is lost. This is why the pipeline runs comfortably in real time on a single webcam feed and CPU.

9.2 The 21 Landmarks

Each detected hand yields 21 landmarks, indexed 0–20, following a fixed skeletal topology from the wrist outward through each of the five digits:

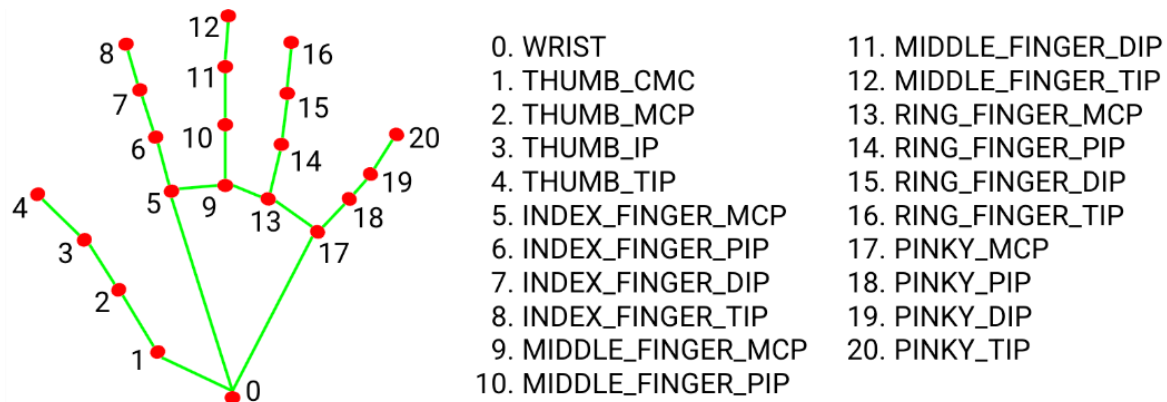


Figure 6: MediaPipe’s 21-point hand landmark topology. Landmark 0 is the wrist; each finger is a chain of 4 landmarks (MCP, PIP, DIP, TIP for the four fingers; CMC, MCP, IP, TIP for the thumb).

For the virtual mouse, three landmarks matter most: **landmark 8** (INDEX_FINGER_TIP) drives cursor position, **landmark 4** (THUMB_TIP) is used with landmark 8 to detect a pinch-to-click gesture, and the tip landmarks of all four fingers (8, 12, 16, 20) compared against their respective PIP joints determine which fingers are extended, used for gesture states such as scroll mode.

9.3 Setting Up the Detector

Listing 17: Initializing MediaPipe Hands and running it on a webcam frame

```

1 import mediapipe as mp
2
3 mp_hands = mp.solutions.hands
4 mp_drawings = mp.solutions.drawing_utils
5 hands = mp_hands.Hands(max_num_hands=1, min_detection_confidence
6     =0.7)
7
8 rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
9 result = hands.process(rgb)
10 if result.multi_hand_landmarks:
11     for hand_landmarks in result.multi_hand_landmarks:
12         mp_drawings.draw_landmarks(frame, hand_landmarks,
13             mp_hands.HAND_CONNECTIONS)

```

`max_num_hands=1` restricts detection to a single hand, which is both sufficient for mouse control and faster than tracking multiple hands. `min_detection_confidence` sets the palm-detector threshold below which a frame is treated as “no hand found.” Note that `Hands.process` expects an **RGB** image while OpenCV captures frames in **BGR**, so every frame must be converted with `cvtColor` before being passed in — the same BGR/RGB distinction encountered in Section 3. Each landmark is accessed as `hand_landmarks.landmark[i]`, exposing `.x`, `.y`, `.z` as floats normalized to `[0, 1]`.

10 End-Term Deliverable: Gesture-Controlled Virtual Mouse

The final deliverable combines OpenCV frame capture, MediaPipe hand tracking, and `pyautogui` system control into a working virtual mouse: index-finger tracking moves the cursor, a thumb-index pinch triggers clicks (with double-click detection), and a four-finger-raised gesture enables scrolling. The full script is walked through block by block below.

10.1 Setup and Configuration

Listing 18: Imports, MediaPipe/pyautogui setup, and tunable constants

```
1 import cv2
2 import numpy as np
3 import mediapipe as mp
4 import pyautogui
5 import time
6 import math
7
8 mp_hands = mp.solutions.hands
9 mp_drawings = mp.solutions.drawing_utils
10 hands = mp_hands.Hands(max_num_hands=1, min_detection_confidence
    =0.7)
11
12 smoothing = 5
13 frame_reduction = 100
14
15 screen_w, screen_h = pyautogui.size()
16 prev_screen_x, prev_screen_y = 0, 0
17 scroll_ref_y = None
18
19 frameWidth = 640
20 frameHeight = 480
21 cap = cv2.VideoCapture(0)
22 cap.set(3, frameWidth)
23 cap.set(4, frameHeight)
24
25 click_start_time = None
26 click_times = []
27 click_cooldown = 0.5
28 scroll_mode = False
29 freeze_cursor = False
```

`pyautogui.size()` reads the actual screen resolution, which is almost always larger than the 640×480 webcam frame, so finger coordinates must later be remapped into screen space. `smoothing` and `frame_reduction` are the two key tuning constants explained below. The state variables (`prev_screen_x/y`, `click.times`, `scroll_mode`, `freeze_cursor`) persist

across frames and are what give the system gesture *memory* rather than reacting to each frame in isolation.

10.2 Reading Landmarks and Detecting Extended Fingers

Listing 19: Per-frame landmark extraction and finger-state detection

```
1 success, frame = cap.read()
2 frame = cv2.flip(frame, 1)
3 rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
4 result = hands.process(rgb)
5 if result.multi_hand_landmarks:
6     for hand_landmarks in result.multi_hand_landmarks:
7         mp_drawings.draw_landmarks(frame, hand_landmarks,
8                                     mp_hands.HAND_CONNECTIONS)
9         thumb_tip = hand_landmarks.landmark[4]
10        index_tip = hand_landmarks.landmark[8]
11        middle_tip = hand_landmarks.landmark[12]
12        ring_tip = hand_landmarks.landmark[16]
13        pinky_tip = hand_landmarks.landmark[20]
14
15        fingers = [
16            1 if hand_landmarks.landmark[tip].y <
17               hand_landmarks.landmark[tip-2].y else 0
18            for tip in [8, 12, 16, 20]
19        ]
```

`cv2.flip(frame, 1)` mirrors the frame horizontally so the on-screen motion matches the user's intuitive left/right hand motion (a webcam feed is otherwise mirror-reversed relative to the user). The **finger-extended test** compares each fingertip's *y*-coordinate to the landmark two joints below it (tip-2, i.e. the PIP joint): since MediaPipe's *y* increases downward and an extended finger points up, the fingertip has a *smaller y* than its PIP joint exactly when the finger is straight. This gives a 4-element binary vector `fingers` (index, middle, ring, pinky), used later to detect the scroll gesture without any additional model or heuristic beyond landmark geometry already available.

10.3 Pinch-to-Click with Debouncing

Listing 20: Thumb-index distance, click, and double-click detection

```
1 dist = math.hypot(thumb_tip.x - index_tip.x,
2                  thumb_tip.y - index_tip.y)
3 CLICK_ENTER = 0.055
4 CLICK_EXIT = 0.07
5
6 if dist < CLICK_ENTER:
7     if not freeze_cursor:
8         freeze_cursor = True
```

```

9         click_times.append(time.time())
10        if len(click_times) >= 2 and \
11            click_times[-1] - click_times[-2] < 0.4:
12            pyautogui.doubleClick()
13            click_times = []
14        else:
15            pyautogui.click()
16    elif freeze_cursor and dist > CLICK_EXIT:
17        freeze_cursor = False

```

The pinch distance is the Euclidean distance between thumb tip and index tip in normalized coordinates, computed with `math.hypot`. Two design choices here solve real problems encountered during testing:

Hysteresis (two thresholds instead of one). Using a single distance threshold caused rapid repeated clicking whenever the pinch distance hovered near the boundary due to hand jitter. Splitting into `CLICK_ENTER` (0.055, must cross to register a click) and a looser `CLICK_EXIT` (0.07, must cross back out to re-arm) creates a dead zone between the two, so a single physical pinch reliably produces exactly one click event rather than several.

freeze_cursor as a click latch. The `if not freeze_cursor` guard ensures a click only fires on the *transition* into the pinch, not on every frame the pinch is held. The same flag also freezes cursor movement during a pinch (see below), preventing the cursor from drifting mid-click as the hand naturally shifts while pinching.

Double-click detection reuses the same `click_times` list: every click timestamp is appended, and if the two most recent clicks are within 0.4s of each other, a `doubleClick()` fires instead of two separate single clicks, and the list is cleared to avoid a stray third click chaining into another double-click.

10.4 Cursor Mapping and Smoothing

Listing 21: Mapping fingertip position to screen space with exponential smoothing

```

1  if not freeze_cursor:
2      raw_x = int(index_tip.x * frameWidth)
3      raw_y = int(index_tip.y * frameHeight)
4      screen_x_target = np.interp(raw_x,
5          [frame_reduction, frameWidth - frame_reduction], [0,
6              screen_w])
7      screen_y_target = np.interp(raw_y,
8          [frame_reduction, frameHeight - frame_reduction], [0,
9              screen_h])
10
11     screen_x = prev_screen_x + (screen_x_target - prev_screen_x) /
12         smoothing

```

```

10     screen_y = prev_screen_y + (screen_y_target - prev_screen_y) /
        smoothing
11     screen_x = min(max(screen_x, 0), screen_w - 1)
12     screen_y = min(max(screen_y, 0), screen_h - 1)
13
14     pyautogui.moveTo(screen_x, screen_y, duration=0)
15     prev_screen_x, prev_screen_y = screen_x, screen_y

```

Three separate mechanisms combine here:

Normalized-to-pixel conversion. `index_tip.x/.y` are normalized to $[0, 1]$ (Section 9), so multiplying by `frameWidth/frameHeight` recovers the fingertip’s pixel position within the 640×480 webcam frame.

frame_reduction and `np.interp`. Rather than mapping the full 640×480 frame to the screen, only the inner region $[100, 540] \times [100, 380]$ (a 100px margin on each side) is mapped to $[0, \text{screen_w}] \times [0, \text{screen_h}]$. This makes the extreme edges and corners of the screen reachable without requiring the hand to leave the webcam’s field of view — without this margin, the cursor could never reach the screen edges since a real hand rarely occupies the very edge pixels of the frame. `np.interp` performs this linear remapping directly.

Exponential smoothing. Rather than jumping straight to `screen_x_target` every frame, the cursor moves only $1/\text{smoothing}$ of the remaining distance each frame: $x_t = x_{t-1} + (x_{\text{target}} - x_{t-1})/5$. This is a first-order low-pass filter that damps the frame-to-frame jitter inherent in landmark detection (a fingertip’s detected position wobbles by a few pixels even when physically still), trading a small amount of input lag for a visibly steadier cursor.

The `if not freeze_cursor` guard means all of this is skipped entirely while a pinch is held, so the cursor position freezes for the duration of a click rather than drifting with natural hand movement.

10.5 Scroll Mode

Listing 22: Four-finger scroll gesture: entering scroll mode and firing scroll events

```

1  if sum(fingers) == 4:
2      if not scroll_mode:
3          scroll_ref_y = index_tip.y
4          scroll_mode = True
5  else:
6      scroll_mode = False
7      scroll_ref_y = None
8
9  if scroll_mode and scroll_ref_y is not None:
10     delta = index_tip.y - scroll_ref_y
11     if delta < -0.08:

```

```

12     pyautogui.scroll(60)
13     if delta > 0.08:
14         pyautogui.scroll(-60)

```

Raising all four fingers (`sum(fingers) == 4`, from the extended-finger vector computed earlier) enters scroll mode and records a **reference *y*-position** at the moment of entry, rather than scrolling based on absolute finger height. Subsequent frames compare the current index-tip *y* against this fixed reference: moving the hand up by more than 0.08 (in normalized units) triggers a positive scroll (up), moving it down by the same margin triggers a negative scroll (down). This relative-displacement design means scrolling works consistently regardless of where in the frame the gesture starts, and resets its reference point every time scroll mode is re-entered, avoiding drift across separate scroll gestures.

10.6 Main Loop

Listing 23: Full per-frame loop and clean shutdown

```

1  while True:
2      success, frame = cap.read()
3      frame = cv2.flip(frame, 1)
4      rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
5      result = hands.process(rgb)
6      if result.multi_hand_landmarks:
7          # ... landmark extraction, click, cursor, scroll logic above
8          ...
9          pass
10     cv2.imshow("Live Feed", frame)
11     if cv2.waitKey(1) & 0xFF == ord('q'):
12         break
13 cap.release()
14 cv2.destroyAllWindows()

```

The structure mirrors the midterm tracker’s main loop (Section 4): read a frame, flip/convert it, run detection, act on the result, display, and check for the quit key every iteration. `cap.release()` and `destroyAllWindows()` again ensure the webcam is not left locked if the script is interrupted.

10.7 Live Demo

Figure 7 shows the completed system running live: MediaPipe’s 21-point skeleton overlaid on a detected hand, with the cursor responding to index-finger movement in real time.

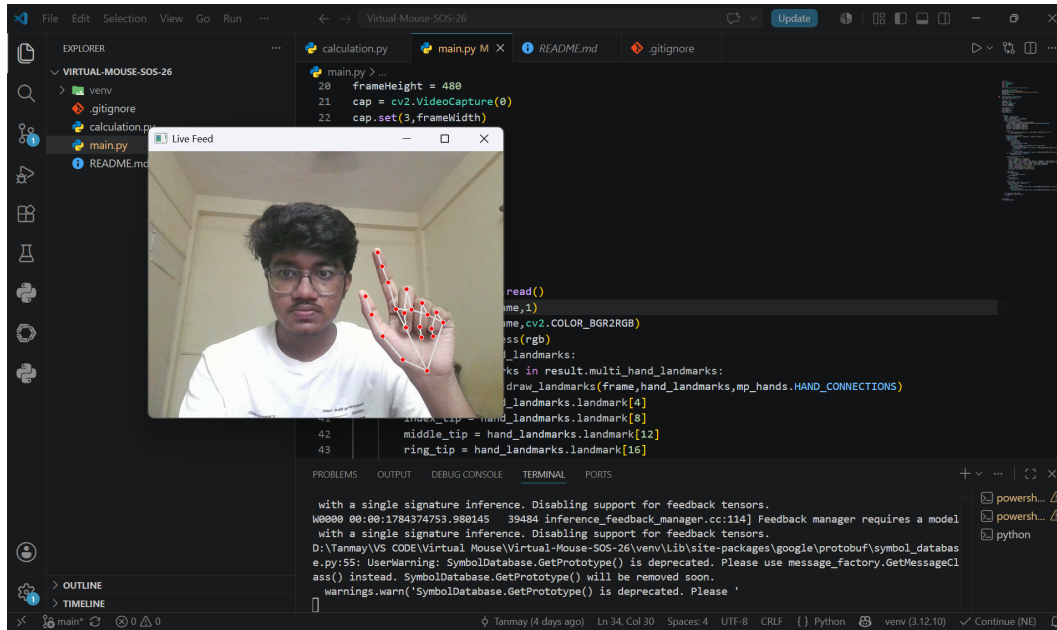


Figure 7: Live virtual mouse: hand landmarks detected and drawn in real time, cursor tracking the index fingertip.

11 Conclusion

Over the course of Summer of Science 2026, this project moved from Python and NumPy fundamentals through the full core OpenCV toolkit — color space conversion, filtering, edge detection, morphological operations, perspective transforms, and contour-based shape analysis — to a working real-time colored object tracker at the midterm checkpoint. The tuning process was a concrete lesson in why live, interactive parameters matter: the target object’s actual HSV range fell in an entirely different band of the color wheel than expected, a gap that would have been very difficult to find without trackbar-driven exploration.

The second half of the project built on that classical CV foundation with modern deep learning. Convolutional Neural Networks (Section 8) explained *why* spatial, weight-shared filters outperform fully-connected layers on images, and MediaPipe Hands (Section 9) provided a production-grade, pre-trained instance of exactly that idea, delivering 21 real-time hand landmarks from a single RGB frame. Combining this landmark stream with the OpenCV frame-capture pipeline from the midterm and `pyautogui` for system control produced the end-term deliverable: a gesture-controlled virtual mouse capable of smooth cursor tracking, debounced pinch-to-click with double-click detection, and a four-finger scroll gesture, all running live on a single webcam feed.

Beyond the formal curriculum, the 3Blue1Brown neural network series was worked through end to end and extended independently into the transformer architecture behind large language models and into diffusion-based image generation. Taken together, the project traces a full arc from raw pixels and NumPy arrays, through classical thresholding and contour geometry, through the weighted-sum-and-backpropagation core of neural networks, into the

convolutional and attention-based architectures that power modern computer vision and generative AI — with the virtual mouse standing as a concrete, hands-on (literally) demonstration of a pre-trained deep learning model integrated into a real, interactive application.

References

1. CS50’s Introduction to Programming with Python, Harvard. <https://cs50.harvard.edu/python>
2. The Python Tutorial, Python Software Foundation. <https://docs.python.org/3/tutorial>
3. NumPy Documentation. <https://numpy.org/doc/stable/>
4. Murtaza Hassan, *OpenCV Python Tutorial for Beginners*, YouTube.
5. OpenCV Python Tutorials (official). https://docs.opencv.org/4.x/d6/d00/tutorial_py_root.html
6. LearnOpenCV, Satya Mallick. <https://learnopencv.com>
7. Grant Sanderson (3Blue1Brown), *Neural Networks* series, YouTube.
8. Grant Sanderson (3Blue1Brown), *Attention in Transformers, visually explained*, YouTube.
9. Stanford CS229, Supervised Learning Lectures 1–4, YouTube.
10. Stanford CS231n: Convolutional Neural Networks for Visual Recognition, Fei-Fei Li et al., YouTube (Lecture 5).
11. Andrew Ng, *Deep Learning Specialization* Course 1, Coursera.
12. PyTorch Official Tutorials. <https://pytorch.org> (60-Minute Blitz).
13. MediaPipe Hands Documentation, Google. <https://developers.google.com/mediapipe>
14. Vaswani et al., “Attention Is All You Need,” NeurIPS 2017. <https://arxiv.org/abs/1706.03762>
15. Ho et al., “Denoising Diffusion Probabilistic Models,” NeurIPS 2020. <https://arxiv.org/abs/2006.11239>
16. Song et al., “Score-Based Generative Modeling through SDEs,” ICLR 2021. <https://arxiv.org/abs/2011.13456>
17. Radford et al., “Learning Transferable Visual Models From Natural Language Supervision” (CLIP), ICML 2021. <https://arxiv.org/abs/2103.00020>